Green TAC: setup and demonstration guide

This guide explains how to use the original project from a **GitHub Download ZIP** or a supplied copy of that ZIP. It covers installing the required tools, opening the source in VS Code, building the compiler, running the examples, and understanding the output.

No previous experience with terminal commands is assumed. Please follow the steps in order on the first run. Commands are provided as they should be entered, and each stage includes a way to check the result.

Green TAC is a C++17 compiler prototype for a small C-like language. It produces Three-Address Code (TAC), compares optimization strategies, and demonstrates carbon-aware strategy selection. Python scripts provide the build and command interface.

Recommended first-use route: Windows with VS Code and its PowerShell terminal. Linux and macOS instructions are in section 10. VS Code is an editor; the C++ compiler must be installed separately.

What is needed

• The complete extracted project ZIP.

• Python 3.10 or later.

• A C++17 compiler available to the terminal: `g++`, `c++`, or `clang++`.

• VS Code for viewing source and terminal output, or another editor and terminal.

• Internet access for downloading tools initially. The basic build and examples run locally once the tools are installed.

Git is not required when using a ZIP. The basic build, tests and examples use Python's standard library; analysis packages are not part of the initial setup.

Reading order

1. Extract the GitHub ZIP.
2. Install the Windows tools.
3. Open the project in VS Code and check the terminal.
4. Build and verify the compiler.
5. Run and understand the two main examples.
6. Read the carbon-aware decision.
7. Edit input and save output.
8. Repeat a short demonstration.
9. Resolve common difficulties.
10. Use another operating system or the optional research tools.

1. Download and extract the project ZIP

Obtain the source

1. Open the Green TAC repository.

2. Select the branch intended for submission, normally `main`.

3. Choose **Code > Download ZIP** and wait for the download to finish.

4. If the ZIP was supplied directly, begin with that file instead. A GitHub account is not needed to run an already supplied ZIP.

Extract the complete ZIP

On Windows, right-click the ZIP, choose **Extract All**, select a normal writable folder, and finish extraction. Open the extracted folder rather than running files from the compressed ZIP preview.

A download from `main` commonly creates `Green-TAC-main`. The name can differ for other branches or if the folder was renamed. Use the actual folder name on your computer.

The important part of the structure is:

```
Green-TAC-main/
  README.md
  SETUP.md
  green-tac/
    build.py
    run.py
    src/
    include/
    benchmarks/
    data/
    tests/
    tools/
```

The **repository root** is the outer folder containing `README.md` and `SETUP.md`. The **working folder** for the commands in this guide is the inner `green-tac` folder containing `build.py` and `run.py`.

Keep the extracted project together. Copying only `run.py`, one input file, or an existing executable is insufficient for the source-build workflow.

Check before continuing: locate `green-tac/build.py`, `green-tac/run.py`, and `green-tac/benchmarks/cse.c` in File Explorer. If they are absent, check for another nested folder or extract the complete ZIP again.

2. Install the Windows tools

Python

Download Python using the instructions at Python for Windows. Install a supported Python 3 version that is at least 3.10. Installer interfaces vary; follow the installation method offered by Python, including its runtime installation prompts.

Open a new PowerShell window and check:

```
python --version
```

Expected: a version such as `Python 3.13.x`, with a version number of 3.10 or newer. If `python` is unavailable but `py --version` works, use `py` wherever this guide says `python`. See the official Windows Python guide if neither command works.

C++ compiler with MSYS2

1. Install MSYS2 following the official MSYS2 instructions. These commands assume the default location, `C:\msys64`.

2. Open **MSYS2 UCRT64** from the Start menu. This is a different terminal from PowerShell.

3. In that UCRT64 terminal, install GCC with the following command. Accept the package installation prompts and wait until the terminal prompt returns.

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

That package command belongs in **MSYS2 UCRT64**, not PowerShell. If you already have a working C++17 compiler, you may use it instead of installing a second compiler.

VS Code

Install the desktop version from Visual Studio Code. Reopen VS Code after installing tools so that new terminals see the updated environment. A C/C++ extension may assist with editing, but it is not required for this project's terminal commands and does not replace GCC.

If your institution manages software installation, ask the IT team to provide Python and a C++17 compiler. Once installed, continue with the checks in section 3.

3. Open VS Code and prepare the terminal

1. Open VS Code and choose **File > Open Folder**.

2. Select the extracted **inner** `green-tac` **folder**, not only a `.c` file. The Explorer panel should show `build.py`, `run.py`, `src` and `benchmarks`.

3. If workspace trust is requested, review the source and follow your institution's policy. Trust only the project copy you recognize.

4. Choose **Terminal > New Terminal**. Use the terminal profile dropdown to choose **PowerShell** if a different shell opens.

Enter the commands in VS Code's **Terminal** panel. Type or paste one command, then press Enter. Do not type the displayed `PS C:\...>` prompt itself.

Make GCC available in this PowerShell session

For the default MSYS2 UCRT64 installation:

```
$env:Path = "C:\msys64\ucrt64\bin;" + $env:Path
python --version
g++ --version
```

The first line affects only this terminal session. Repeat it when opening a new terminal unless the compiler folder is already on your user PATH. GCC's matching runtime DLLs must also remain available when running the built compiler.

For an existing MSYS2 **MINGW64** compiler, replace `ucrt64` with `mingw64`. If MSYS2 is installed elsewhere, use its actual compiler folder. Use the directory belonging to the compiler you installed.

Confirm the working folder

```
Get-Location
Test-Path .\build.py
Test-Path .\run.py
```

Both `Test-Path` commands should print `True`. If you opened the outer repository folder in VS Code instead, run `cd green-tac` once and repeat the checks. If both are already `True`, do not run `cd green-tac` again.

Ready to build: Python reports version 3.10+, `g++ --version` prints compiler information, and both script checks return `True`.

4. Build and verify the compiler

Run these commands from the inner `green-tac` folder, one at a time:

```
python build.py
python run.py test
python -m unittest discover -s tests -p "test_*.py"
```

What each command does

1. `python build.py` compiles C++ sources in `src` using headers in `include`. Successful output ends with `Built:` and the executable path. Compilation may take some time; wait for the prompt to return.

2. `python run.py test` rebuilds the compiler and C++ test executable, then runs the C++ tests. Look for `All tests passed.` A second compilation here is expected.

3. **The unittest command** runs the Python regression and research-evidence checks. Look for `OK`. Evidence-dependent tests may be skipped if optional research data is absent; the report identifies skipped tests.

The resulting compiler is `bin/green_tac.exe` on Windows and `bin/green_tac` on Linux or macOS.

Check a command's exit status

Immediately after a command finishes, enter this in PowerShell:

```
$LASTEXITCODE
```

`0` means that the preceding external command completed successfully. A nonzero value means it reported a failure. Read the earlier error before continuing, and use section 9 to resolve it.

When to rebuild

Build after extracting the ZIP, even if it already contains an executable. That executable may have been built on a different system or from different source files.

Build again whenever you change C++ code in `src` or headers in `include`. The runner automatically builds only if the executable is absent; it does not check whether an existing binary is older than the source.

Changing an input under `benchmarks` does **not** require rebuilding the compiler. Save the input file and rerun the appropriate command.

The basic build does not require CMake, Make, a Python virtual environment, or third-party Python packages.

5. Run the examples and understand the output

Example A: Boolean conditions

Open `benchmarks/cse.c` in VS Code's editor. In the terminal, run:

```
python run.py steps benchmarks/cse.c
python run.py compare benchmarks/cse.c
```

The `steps` command shows the saved source, tokens, Abstract Syntax Tree (AST), baseline TAC, optimization details and final comparison. Scroll upward to begin with the source. Enlarge the terminal or widen the window if a table wraps.

Tokens are pieces of the program, such as names and operators. The AST represents its structure. TAC breaks expressions into simple instructions. Labels and branches represent control flow. Short-circuit evaluation skips the right side of AND when the left side is false, and skips the right side of OR when the left side is true.

The current Boolean example contains **22 TAC instructions in all three strategies**. No reduction is expected for this input; it is primarily a control-flow demonstration.

Example B: optimization and a while loop

Open `benchmarks/optimization_showcase.c`, then run:

```
python run.py steps benchmarks/optimization_showcase.c
python run.py compare benchmarks/optimization_showcase.c
```

The comparison for the bundled source is:

```
Strategy       TAC instructions    Binary operations
baseline              30                  13
lightweight           30                   9
dag                   27                   6
```

Look for constant folding (`10 + 20` becomes `30`), algebraic simplification (multiplication by 1), copy propagation, and common-subexpression elimination (reusing repeated expressions such as `p + q` when safe). DAG adds local sharing of repeated expressions to the lighter transformations.

The DAG reduction is 3 out of 30 instructions, or **10%**. These are static code counts, not measurements of speed or energy savings. A loop may execute an instruction many times even though it appears once in the listing.

Use these commands rather than a generic **Run Code** or C/C++ debug button. The inputs use Green TAC's small language, not complete C programs with `main`. Normal output describes compiler stages; it does not print the final numerical value of every variable.

6. Run the carbon-aware selection

From the same folder, run this PowerShell command. Copy all three lines together; each backtick continues onto the next line, with no spaces after the backtick:

```
python run.py green benchmarks/optimization_showcase.c `
  --compile-ci 700 --runtime-ci 200 `
  --executions 10000
```

Alternatively, enter it as one continuous line without the backticks. On Linux or macOS, use a single line or your shell's continuation syntax.

Understand the inputs

- `--compile-ci 700`: assumed carbon intensity during compilation.
- `--runtime-ci 200`: assumed carbon intensity during execution.
- `--executions 10000`: expected number of program executions.

Carbon intensities are in grams CO2e per kilowatt-hour. These are hypothetical demonstration assumptions, not live grid readings. The default model is `data/model.csv`; the default allowed predicted runtime overhead is 10%.

Read the output in order

1. **Model status:** identifies available models and whether proxy estimates are used.
2. **Candidate table:** compares baseline, lightweight and DAG. `Compile J` is compilation energy, `Runtime J` is energy per execution, `Runtime ms` is predicted time, and `CO2e (g)` is modeled total carbon.
3. **Allowed:** indicates whether the candidate meets the predicted runtime constraint.
4. **Selected strategy:** identifies the chosen eligible strategy, followed by its TAC.

Compilation has a cost, while optimization may reduce repeated execution costs. The selector balances them using the expected execution count and the two carbon intensities.

Interpretation: missing energy models use explicitly labeled **PROXY** estimates. These are not measured energy or carbon savings. The fitted runtime model has poor documented holdout accuracy; its predicted runtime limit is not a measured performance guarantee.

For the project's fixed adaptive-selection scenarios, run:

```
python run.py adaptive-demo
```

Those scenarios use `benchmarks/adaptive_reference.c`. They demonstrate selector behavior and are separate from edits to the two walkthrough inputs.

7. Edit input and save the output

Create a small example

In VS Code, create a file and save it as `benchmarks/my_example.c` with this content:

```
a = 10 + 20;
b = a * 1;
c = b + 0;
```

Press **Ctrl+S** before running. Use UTF-8 without a byte-order mark where an encoding choice is available. Check that the filename is `.c`, not `.c.txt`.

```
python run.py steps benchmarks/my_example.c
python run.py tac benchmarks/my_example.c --strategy dag
```

The `steps` output starts with the saved source that was read. If an edit seems to have no effect, check that section and your command's file path. Editing a benchmark does not require rebuilding.

Supported input

Use integer arithmetic, assignments, comparisons, Boolean expressions, `if`/`else`, and `while`. Braces clearly group statement bodies; single-statement bodies are also supported. Do not add `#include`, a `main()` function, or variable declarations to these examples. `for`, `do/while`, `break`, and `continue` are not implemented.

Save a comparison from PowerShell

This command displays the output and saves a copy in the working folder. Copy both lines together; the pipe continues the command:

```
python run.py compare benchmarks/optimization_showcase.c |
  Tee-Object -FilePath comparison-output.txt
```

Open `comparison-output.txt` in VS Code's Explorer panel. Repeating the command overwrites that file. Choose a different name to preserve multiple runs.

To capture errors as well as normal compiler output:

```
python run.py steps benchmarks/my_example.c 2>&1 |
  Tee-Object -FilePath my-example-log.txt
```

When seeking assistance, share the exact command, relevant source and error text. You do not need to share unrelated files.

8. Repeat a short demonstration

Once the first build and tests succeed, this sequence is suitable for a review session. All commands run from the inner `green-tac` folder.

Prepare the session

Open that folder in VS Code, create a PowerShell terminal and make the compiler available again if needed:

```
$env:Path = "C:\msys64\ucrt64\bin;" + $env:Path
python --version
g++ --version
```

Use your actual compiler directory. A fresh download or a change to compiler source requires `python build.py` before proceeding.

Demonstrate the project

1. Open `benchmarks/cse.c` in the editor, then show its compiler stages:

```
python run.py steps benchmarks/cse.c
```

2. Open `benchmarks/optimization_showcase.c`, then compare the strategies:

```
python run.py compare benchmarks/optimization_showcase.c
```

3. Explain the change from 30 to 27 instructions and from 13 to 6 binary operations. Use `steps` on this input when a detailed transformation trace is required.

4. Run the carbon-aware command from section 6. Read the assumptions and proxy label before discussing the selected strategy.

5. Save the comparison using section 7 if a text record is needed.

Completion checklist

• The source builds successfully on the receiving computer.

• C++ tests report `All tests passed.` and Python tests report `OK`.

• The source shown by `steps` matches the intended saved input.

• Example counts agree with section 5 when bundled inputs are unchanged.

• Compiler output is visible in VS Code's Terminal panel.

• Proxy estimates are distinguished from physical measurements.

A concise explanation is: "The compiler shows how source becomes TAC and how optimizations change that representation. Carbon-aware selection compares modeled costs under stated assumptions. Actual energy savings require separate measurements."

9. Troubleshooting

Python is not recognized or opens an installation prompt

Finish installing Python, then reopen VS Code and its terminal. Try `py --version`; if it works, replace `python` with `py`. Confirm version 3.10 or newer.

C++ compiler not found, or g++ is not recognized

Run `g++ --version` in the **same terminal** used for building. Add the actual MSYS2 compiler folder to that terminal's PATH as shown in section 3. A VS Code extension does not install GCC.

A MinGW DLL is missing, or the compiler closes immediately

Keep the matching `ucrt64/bin` or `mingw64/bin` on PATH and rebuild. Use `python run.py ...` in the terminal instead of double-clicking the compiler; diagnostic messages remain visible there.

Python cannot open build.py or run.py

Check `Get-Location` and `Test-Path .\build.py`. If you are in the outer repository folder, run `cd green-tac`. If the scripts remain absent, check the extracted ZIP structure.

The compiler reports a syntax error

Check the reported line or token, semicolons, parentheses and braces. Remove accidental pasted bullets or Markdown fences. Try section 7's three-line example. The optimization example should begin with `a = 10 + 20;`.

Output is incorrect or edits are ignored

Save with Ctrl+S, check the command's input path and inspect the source shown by `steps`. Rebuild after C++ or header changes. Use `python run.py`; a generic C/C++ run button invokes a different workflow.

No instruction reduction or physical energy reading appears

The Boolean example keeps 22 instructions. Use the optimization example to show a reduction. This project's physical RAPL energy measurement is unavailable on Windows; proxy labels and unavailable energy information are expected.

A test fails or output is hard to read

Do not treat failing tests as a successful setup. Keep the full message for the project author. For long output, enlarge the terminal, scroll up, or save text using section 7. OUTPUT and DEBUG CONSOLE are different panels from the Terminal used here.

10. Other systems and optional research work

Linux: Ubuntu or Debian

Install the tools, extract the ZIP and enter its inner `green-tac` folder:

```
sudo apt update
sudo apt install python3 python3-venv g++ gcc
python3 --version
g++ --version
```

Use `python3` instead of `python` throughout this guide. Omit the Windows PATH command. Build with `python3 build.py`, then run `python3 run.py test` and the examples. Git is unnecessary for a ZIP.

macOS

Install Apple's command-line tools with `xcode-select --install`. Install Python 3.10+ from Python for macOS if needed. Check `python3 --version` and `clang++ --version`, then enter the extracted inner `green-tac` folder and use `python3`. The build detects `c++` or `clang++` when `g++` is unavailable.

Optional analysis environment

The basic compiler and demonstrations do not need extra Python packages. Analysis, plots and model fitting require dependencies and suitable measurement data. On Windows, from `green-tac`:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r tools/requirements.txt
```

On Linux/macOS, use `.venv/bin/python` instead. Calling that Python directly avoids needing to activate the environment. Follow RESEARCH_WORKFLOW.md (green-tac/RESEARCH_WORKFLOW.md) before collecting or analyzing measurements; this is separate from the basic setup.

References and further reading

• Project command reference (green-tac/README.md)

• Research workflow and evidence limitations (green-tac/RESEARCH_WORKFLOW.md)

• Official Python installation guidance for Windows

• Official MSYS2 installation and GCC setup

• VS Code integrated terminal guide

This guide describes the repository's source-build workflow. The PDF edition is generated from this same Markdown document so that the instructions match.